

Outline

The problem: training error lies

Train/test split

Why train/test isn't enough: validation

Cross-validation

Stratified CV (and friends)

Wrap-up

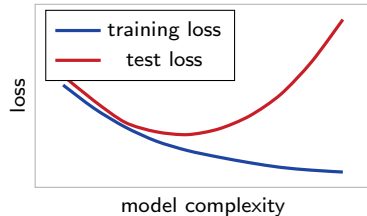
Why we need to hold data out

We fit a model by minimizing empirical risk on the training data (L01, L01b). The fitted model is the one with the lowest training loss.

Problem: the training loss is also the thing we used to choose the model. We're *grading the model on the questions we wrote the answer key from*.

We need an **independent estimate** of how the model will do on data it hasn't seen.

The picture every ML student should have in mind:

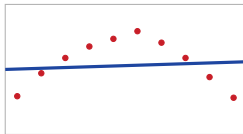


Training loss keeps falling. Test loss has a sweet spot, then climbs.

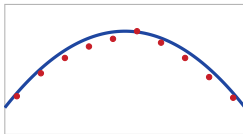
Predict-first: can a degree-9 polynomial fit 10 points perfectly?

You have 10 noisy samples from $y = \sin(\pi x) + \varepsilon$. You fit polynomials of degree 1, 3, and 9. Reminder from L01b:

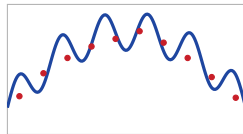
degree 1



degree 3



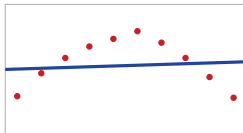
degree 9



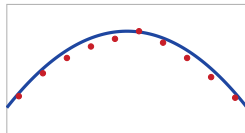
Predict-first: can a degree-9 polynomial fit 10 points perfectly?

You have 10 noisy samples from $y = \sin(\pi x) + \varepsilon$. You fit polynomials of degree 1, 3, and 9. Reminder from L01b:

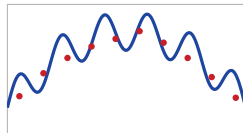
degree 1



degree 3



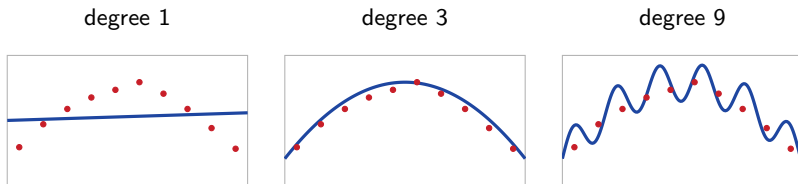
degree 9



Predict: which has the smallest *training* error? Is that the model you should deploy?

Predict-first: can a degree-9 polynomial fit 10 points perfectly?

You have 10 noisy samples from $y = \sin(\pi x) + \varepsilon$. You fit polynomials of degree 1, 3, and 9. Reminder from L01b:



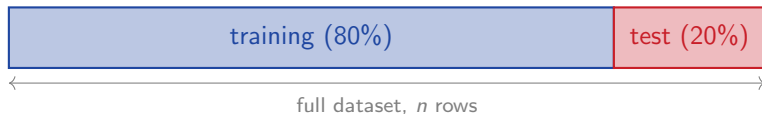
Predict: which has the smallest *training* error? Is that the model you should deploy?

Degree 9 has the smallest training error (it interpolates every point). It is also the *worst* model. Training error is not a measure of model quality.

The fix: hold out data the model never saw

Split the data into two pieces:

- ▶ **Training set** - used to fit the model.
- ▶ **Test set** - used *once* at the very end to estimate generalization.



Typical splits: 80/20, 70/30 for small data, 90/10 for large data.

Tradeoff: smaller test set $\Rightarrow$ more data to train on, but noisier estimate. Bigger test set $\Rightarrow$ honest estimate, but less training data. The right ratio depends on how much data you have.

Mechanics: sklearn train_test_split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,           # fraction held out
    random_state=509,        # default seed (see CLAUDE.md)
    stratify=y,              # classification only - see Section 5
    shuffle=True,            # default True; disable for time series
)
# >>> X_train.shape, X_test.shape
# ((160, 5), (40, 5))
```

- ▶ `test_size=0.2` $\Rightarrow$ 80/20 split.
- ▶ `random_state` fixes the seed - same split every run. Without it, debugging becomes impossible.
- ▶ `stratify=y` for classification: preserves the class ratio. Formalized later as Stratified CV.
- ▶ `shuffle=False` for time-series: keep chronological order (you can't train on tomorrow to predict today).

Critical rules

1. **Split before you do anything else.** Especially before scaling, imputing, encoding. The leakage rule from L01c (*fit on train, transform on train and test*) requires you to have split first.
2. **Don't iterate on the test set.** If you make modeling decisions based on test-set behavior, the test set becomes part of training. Look at test *once*, at the end.
3. **Time series needs chronological splits.** A random shuffle leaks the future into the past. Use `TimeSeriesSplit` or a single chronological cutoff.
4. **Group-structured data needs group splits.** If you have 10 photos per person and randomly split, the same person appears in both train and test. The model memorizes faces, not features. Use `GroupShuffleSplit`.

What about EDA? Distribution-level facts (sizes, classes present, ranges) are fine to check on the full data before splitting. The rule is about *modeling decisions* and *performance numbers*, not about awareness.

The hyperparameter tuning trap

After splitting, you try several models or hyperparameter settings. You evaluate each on the test set, then pick the best.

Common workflow. Let's check whether the reported number is honest.

Each comparison *uses* the test set. If you pick the configuration with the best test score, you've absorbed the test set into your model-selection process.

Analogy. You take a practice exam 100 times until you score perfectly. Do you expect to score perfectly on the real exam?

Predict-first: tuning on the test set

You try 50 different polynomial degrees on the same train/test split. You pick the degree with the smallest test MSE and report that MSE as your model's quality.

Predict-first: tuning on the test set

You try 50 different polynomial degrees on the same train/test split. You pick the degree with the smallest test MSE and report that MSE as your model's quality.

Predict:

- ▶ Is the reported MSE an unbiased estimate of how the model will perform on *new* unseen data?
- ▶ By roughly how much does the reported MSE under-estimate the true generalization error?

Predict-first: tuning on the test set

You try 50 different polynomial degrees on the same train/test split. You pick the degree with the smallest test MSE and report that MSE as your model's quality.

Predict:

- ▶ Is the reported MSE an unbiased estimate of how the model will perform on *new* unseen data?
- ▶ By roughly how much does the reported MSE under-estimate the true generalization error?

Answers:

- ▶ **No.** The chosen configuration happened to do well on *this specific test set*. Some of that performance is noise that won't reproduce on truly new data.
- ▶ **Typically 10-40% optimistic** on small-to-medium datasets when tuning over ~ 50 configurations. The bias grows with the number of comparisons and shrinks with test-set size. On a fresh test set the reported model would score worse than reported.

Fix: use a separate **validation set** for tuning. Touch the test set only once.

The three-way split

Split the data into *three* pieces.



Workflow:

1. Fit each candidate model on **training**.
2. Evaluate each on **validation**. Pick the best.
3. Optionally *refit* the chosen configuration on training + validation (more data = better fit).
4. Evaluate **once** on **test**. Report that number.

Step 3 is common but not universal - some practitioners skip the refit and report the model that was actually tuned on validation. Both views are defensible.

The discipline: test set looked at exactly once

The hardest part isn't the code. It's the discipline:

- ▶ Don't make modeling decisions based on test scores.
- ▶ Don't engineer features after seeing test set behavior.
- ▶ Don't iterate on hyperparameters after seeing the test score.
- ▶ Don't compare models on test and pick the best.

Treat the test set like a sealed envelope you open at the end. Awareness of the test set (sizes, class balance) is fine - making decisions based on it is not.

Kaggle terminology: the test set is the "private leaderboard". Touch it more than once during development and your real-world performance will lag your reported performance.

One validation set is noisy

You have 200 Yerevan listings. 60/20/20 split $\Rightarrow$ 40 validation rows.

Model A scores 0.81 MSE on validation, model B scores 0.79. Is B really better, or is the gap just noise on 40 samples?

Same data, different random split can flip which model looks better. On a 200-row dataset the validation score has a standard error of several percent.

Solution: use k different validation sets and average their scores. Averaging k roughly-independent estimates reduces the variance of the mean by a factor of $\approx 1/k$ (CLT-style, math module 20). This is **k -fold cross-validation**.

k -fold cross-validation: how it works

Split the (non-test) data into k equal chunks called *folds*. For each of k iterations, hold out one fold as validation and train on the rest.



Subtle point: you train k different models (one per iteration), not one. The CV score estimates how well the *training protocol* generalizes, not any specific model's performance. The final model you ship is usually fit on *all* the training data after CV picks the hyperparameters.

Choosing k and the cost

Standard choices:

- ▶ $k = 5$ - the default. Gives 80/20 train/val per fold (matches standard hold-out), bias is low, compute manageable.
- ▶ $k = 10$ - lower variance, 2x more compute. Use when you can afford it.
- ▶ $k = n$ - leave-one-out (LOO). Train n models. Used only when n is tiny; the LOO score has surprisingly high variance because folds are nearly identical.
- ▶ `RepeatedKfold(n_splits=5, n_repeats=3)` - repeats k -fold with different random splits and averages. Lower variance when compute allows.

When CV is worth it:

- ▶ Small datasets ($n < 10,000$) - one validation set is noisy.
- ▶ Model selection / hyperparameter tuning.
- ▶ Reporting a confidence interval on the score (mean $\pm$ std over folds).

When a single split is enough:

- ▶ Big data ($n > 10^6$) - one split is already low-noise.
- ▶ Quick prototyping iterations.
- ▶ Compute is the bottleneck.

CV in code

sklearn footgun: `cross_val_score` *maximizes* by convention. Error metrics are negated (`neg_mean_squared_error`). Always flip the sign when reporting.

```
from sklearn.model_selection import KFold, cross_val_score
from sklearn.linear_model import Ridge

kf = KFold(n_splits=5, shuffle=True, random_state=509)
scores = cross_val_score(
    Ridge(alpha=1.0), X_train, y_train,
    cv=kf,
    scoring="neg_mean_squared_error",
)
print(f"MSE = {-scores.mean():.3f} +/- {scores.std():.3f}")
```

- ▶ Pass `X_train`, not the full dataset - test set stays untouched.
- ▶ For out-of-fold predictions (used in stacking, calibration): `cross_val_predict` returns one prediction per row.
- ▶ Mean and std across folds give a sanity check on stability.

The imbalanced-data problem

Many real classification tasks are imbalanced:

- ▶ Credit-card fraud: 0.2% fraudulent transactions
- ▶ Disease screening: 1% positives in screened population
- ▶ Click-through prediction: 2% click rate
- ▶ Apartment rent fraud detection: maybe 5% suspicious listings

Random k -fold CV draws folds at random. With a 5% positive class, the positives are scattered uniformly - but *uniformly* doesn't mean *evenly*. Some folds get more, some fewer, sometimes zero.

Why care? A fold with 0% positives breaks the protocol in two ways: the model trained without it never sees the rare class, and the score from that fold isn't meaningful.

Next frame: how often does this actually happen?

Predict-first: 5-fold CV on 5% positives

You have 200 rows of which 10 are positive (5%). You run vanilla 5-fold CV.

Predict-first: 5-fold CV on 5% positives

You have 200 rows of which 10 are positive (5%). You run vanilla 5-fold CV.

Predict:

- ▶ How many positives does each fold get *on average*?
- ▶ What's the probability that at least one fold has zero positives?

Predict-first: 5-fold CV on 5% positives

You have 200 rows of which 10 are positive (5%). You run vanilla 5-fold CV.

Predict:

- ▶ How many positives does each fold get *on average*?
- ▶ What's the probability that at least one fold has zero positives?

Answers:

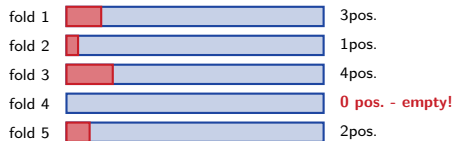
- ▶ **On average 2 positives per fold** (10 split across 5).
- ▶ **Roughly 6% chance** at least one fold is empty - the 10 positives can cluster by chance. With smaller imbalance (1% positives, $n = 200$) it's nearly guaranteed.

Solution: split each class separately, so every fold ends up with its share of the minority class. **Stratified k -fold**.

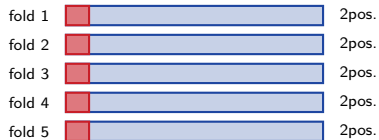
Stratified k -fold: vanilla vs stratified

Same data, two CV strategies. Each row is one fold's positive count (out of 10).

Vanilla k -fold (random)



Stratified k -fold



Stratified k -fold splits each class separately. Every fold receives its share. No more empty-positive folds.

Stratified CV in code

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.linear_model import LogisticRegression

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=509)
scores = cross_val_score(
    LogisticRegression(class_weight="balanced"), # separate trick
    X_train, y_train,
    cv=skf,
    scoring="f1", # not accuracy
)
print(f"F1 = {scores.mean():.3f} +/- {scores.std():.3f}")
```

- ▶ Use StratifiedKFold *by default* for classification. Almost no reason to use plain KFold.
- ▶ Pair with the right metric. F1 / ROC-AUC / PR-AUC handle imbalance honestly; accuracy doesn't.
- ▶ `class_weight="balanced"` is a *model* parameter (orthogonal to stratification) that tells the fit step to weight rare classes more. Commonly paired with stratified splitting on imbalanced data.

When rows aren't independent: group and time splits

Yerevan rent example: you have 200 listings, but only 50 unique landlords. Each landlord has ~ 4 listings on average. A random split puts the same landlord in train AND validation - the model memorizes landlord style instead of learning rent drivers. CV score looks great; live performance is terrible.

Two general cases:

Grouped data. Multiple rows per entity (listings per landlord, reviews per user, photos per person, sensor readings).

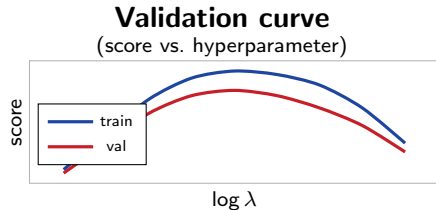
```
from sklearn.model_selection import GroupKFold
GroupKFold(n_splits=5).split(X, y, groups=landlord_id)
```

Time-ordered data. Today's prediction depends on the past, not the future. Random shuffle leaks future into past.

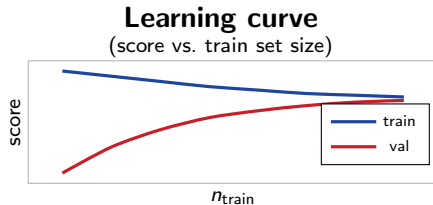
```
from sklearn.model_selection import TimeSeriesSplit
TimeSeriesSplit(n_splits=5)           # rolling-forward windows
```

Validation curves and learning curves

Two CV-based diagnostics every ML practitioner uses. Both plot CV score vs. a single axis.



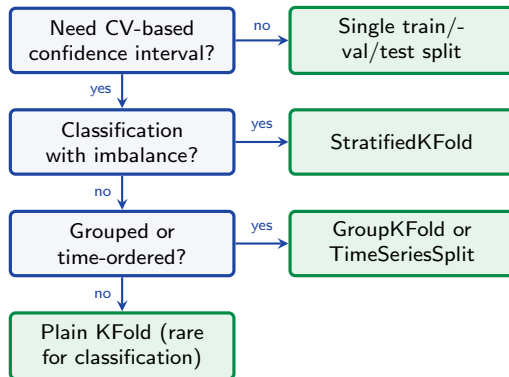
Diagnoses: under/overfitting per hyperparameter setting.



Diagnoses: would more data help? (Gap closing = yes.)

Both available as `sklearn.model_selection.validation_curve` and `learning_curve`.

Decision flowchart



Single train/val/test split is fine when big data makes one split low-noise, when compute is tight, or when you don't need a CI on the score. CV is for when score variance matters.

Recap

- ▶ **Training error lies.** Always evaluate on data the model didn't see.
- ▶ **Train/test split.** Simplest fix. 80/20 typical. Split before any preprocessing (L01c leakage rule).
- ▶ **Validation set.** If you tune anything, you need a third set. Test set touched once at the end.
- ▶ **Cross-validation.** Rotates validation through k folds, reducing noise by $\approx 1/k$. Default $k = 5$. Use for small/medium data and model selection.
- ▶ **Stratified CV.** Default for classification - preserves class ratio per fold.
- ▶ **Group / Time splits.** Use when rows aren't independent. Otherwise the model memorizes entities or peeks at the future.
- ▶ **Validation/learning curves.** CV-based diagnostics for under/overfitting and "would more data help?".

Coming next: L05b - nested CV, the honest way to report a tuned model's generalization error.

HW01d - practice

Estimated time: 60-90 minutes. Tasks 1-3 graded; bonuses optional.

1. On the Yerevan rent dataset: 80/20 split with `random_state=509`. Fit Ridge on train. Report test MAE.
2. Add a validation set (60/20/20). Tune Ridge $\alpha \in \{0.01, 0.1, 1, 10, 100\}$ using validation MSE. Refit best on train+val, report test MAE.
3. Replace step 2's tuning with 5-fold CV on the train set. Compare the chosen α and the std of CV scores.
4. *Bonus 1*: pick a classification task (e.g., is-this-rent-suspicious binary label). Show that `KFold` can produce an empty-positive fold; `StratifiedKFold` doesn't.
5. *Bonus 2*: simulate the tuning trap. Try 50 random hyperparameter configurations, score each on test, pick the best, report the score. Repeat on a fresh test set and report the gap. **Most instructive question.**
6. *Bonus 3*: if your data has a `landlord_id`, run `GroupKFold` on it. Compare to vanilla `KFold`.
7. *Bonus 4*: plot a learning curve for Ridge using `learning_curve`. Discuss whether more data would help.